SIMATS ENGINEERING

Assessment Tool 3 – Coding Challenge

Course: Embedded Systems (ECA1406) | CO2 – BL4

Name of the Student: MOHAMMED THAMEEMUL ANSARI A

Register Number: 192512397

Question No: 28

Title: Comparing Polling-Based Delay and Interrupt-Based Timing in Embedded C (8051)

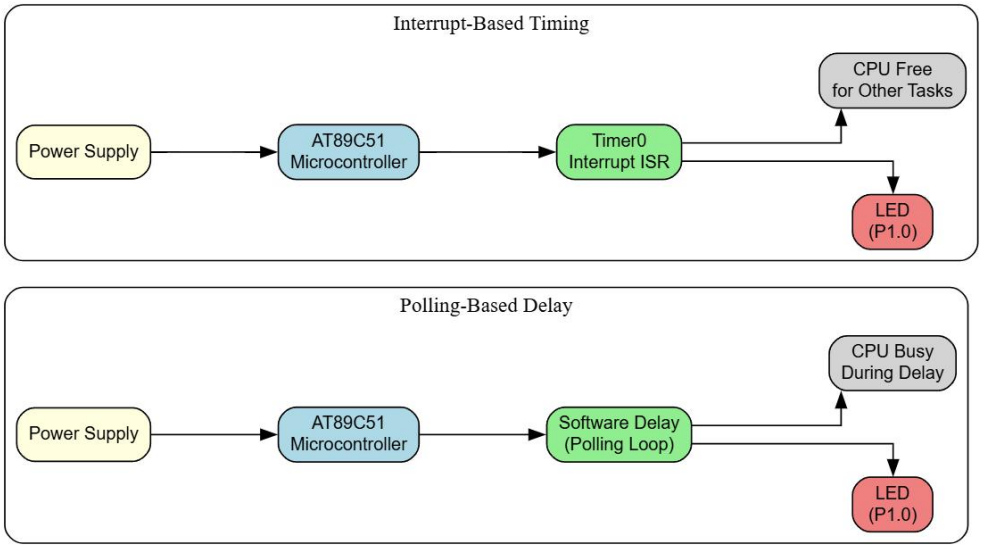
1. Objective

To design, implement, and compare two Embedded C programs on the 8051 microcontroller that toggle an LED at a fixed rate — one using a software (polling-based) delay loop, and the other using a hardware Timer0 interrupt — and to analyze the difference in CPU utilization, timing accuracy, and suitability for real-time embedded applications.

2. Problem Statement

Construct a program comparing polling-based delay and interrupt-based timing in Embedded C. The program should demonstrate LED toggling using (a) a CPU-blocking nested-loop delay, and (b) a Timer0 interrupt service routine (ISR), and evaluate which approach is better suited for real-time industrial systems.

3. Block Diagram



4. Algorithm

Program 1 – Polling-Based Delay

- Start and configure P1.0 as the LED output pin.
- Enter an infinite while(1) loop.
- Toggle the LED state (LED = ~LED).
- Call delay(): run a nested for loop (outer 500, inner 1275 iterations) that keeps the CPU busy for a fixed time.
- Repeat the toggle-and-delay cycle indefinitely; the CPU cannot do any other work while inside delay().

Program 2 – Interrupt-Based Timing (Timer0)

- Configure Timer0 in Mode 1 (16-bit timer) by setting TMOD = 0x01.
- Load initial count values TH0 = 0xFC, TL0 = 0x66 to set the overflow period.
- Enable Timer0 interrupt (ET0 = 1) and global interrupts (EA = 1); start the timer (TR0 = 1).
- Enter an infinite while(1) loop where the CPU is free to perform other tasks.
- On every Timer0 overflow, the ISR Timer0_ISR() executes automatically: it reloads TH0/TL0 and toggles the LED.
- The CPU only branches to the ISR when the timer overflows, and returns to main() immediately after.

4. Coding Approach

Both programs are written in Embedded C for the 8051 architecture using the reg51.h header and simulated/built in Keil µVision 5 (C51 toolchain). The LED is mapped to bit P1.0 using the sbit keyword. The polling approach uses a purely software-timed delay function called from an infinite loop, so the delay duration depends on the CPU clock and instruction cycle count, and the CPU remains occupied for the entire delay period. The interrupt-based approach instead configures Timer0 as a hardware countdown; the toggle logic is moved into an ISR (interrupt 1) so that main() only starts the timer once and then sits in an empty while(1) loop, leaving the CPU free for other tasks between interrupts. Both projects were compiled and run in the Keil simulator, with breakpoints placed to verify timer reload values, register states, and LED toggling behaviour.

5. Source Code

5.1 Program 1 – Polling-Based Delay

```
#include <reg51.h>
sbit LED = P1^0;
void delay()
{
   unsigned int i, j;
   for(i = 0; i < 500; i++)
   {
      for(j = 0; j < 1275; j++);
   }
}
void main()
{
   while(1)
   {
      LED = ~LED;   // Toggle LED
      delay();      // Polling delay
   }
}
```

5.2 Program 2 – Interrupt-Based Timing (Timer0)

```
#include <reg51.h>
sbit LED = P1^0;
void Timer0_ISR(void) interrupt 1
{
   TH0 = 0xFC;      // Reload Timer0
   TL0 = 0x66;
   LED = ~LED;      // Toggle LED
}
void main()
{
```

```
    TMOD = 0x01;     // Timer0 Mode1 (16-bit)
    TH0 = 0xFC;      // Initial timer values
    TL0 = 0x66;
    ET0 = 1;         // Enable Timer0 interrupt
    EA = 1;          // Enable Global Interrupt
    TR0 = 1;         // Start Timer0
    while(1)
    {
       // CPU can perform other tasks here
    }
}
```

6. Output / Screenshots

6.1 Program 1 – Polling-Based Delay (Source in Keil µVision)

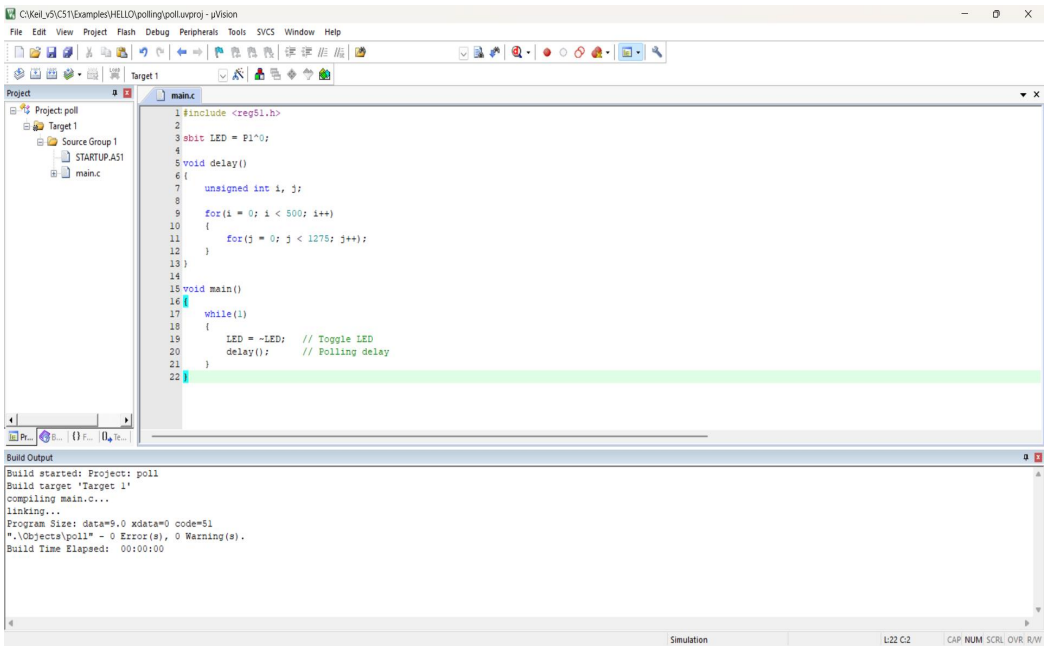


Fig 1: Polling-based delay program – main.c in Keil µVision

6.2 Program 1 – Simulation / Debug Run

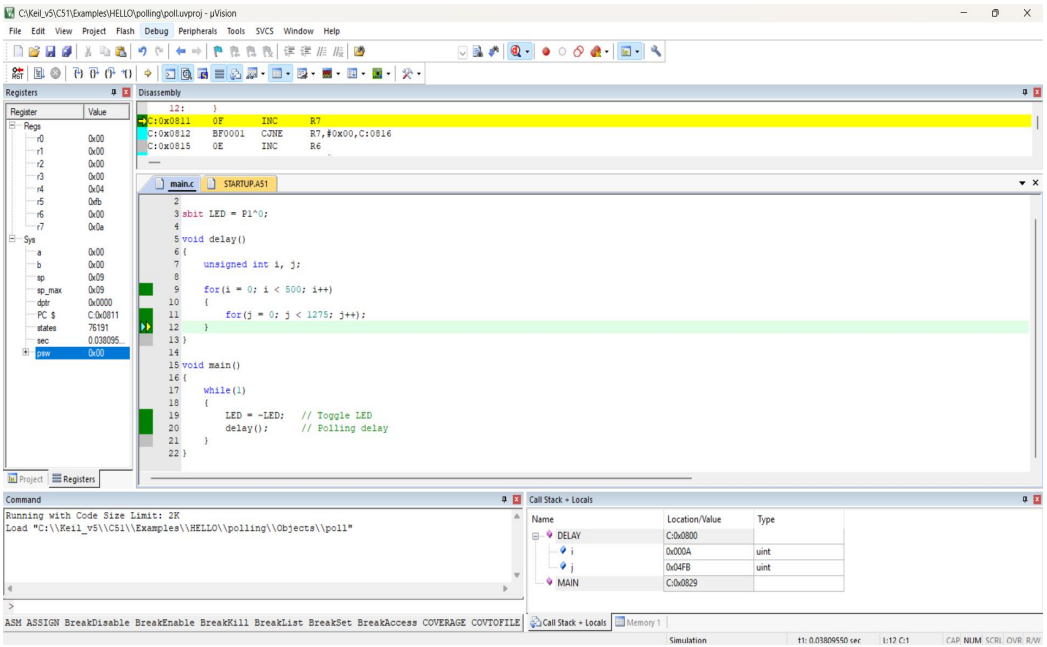


Fig 2: Debug session showing delay() loop executing with CPU busy (R6/R7 counters, PC inside delay loop)

6.3 Program 2 – Interrupt-Based Timing (Source in Keil µVision)

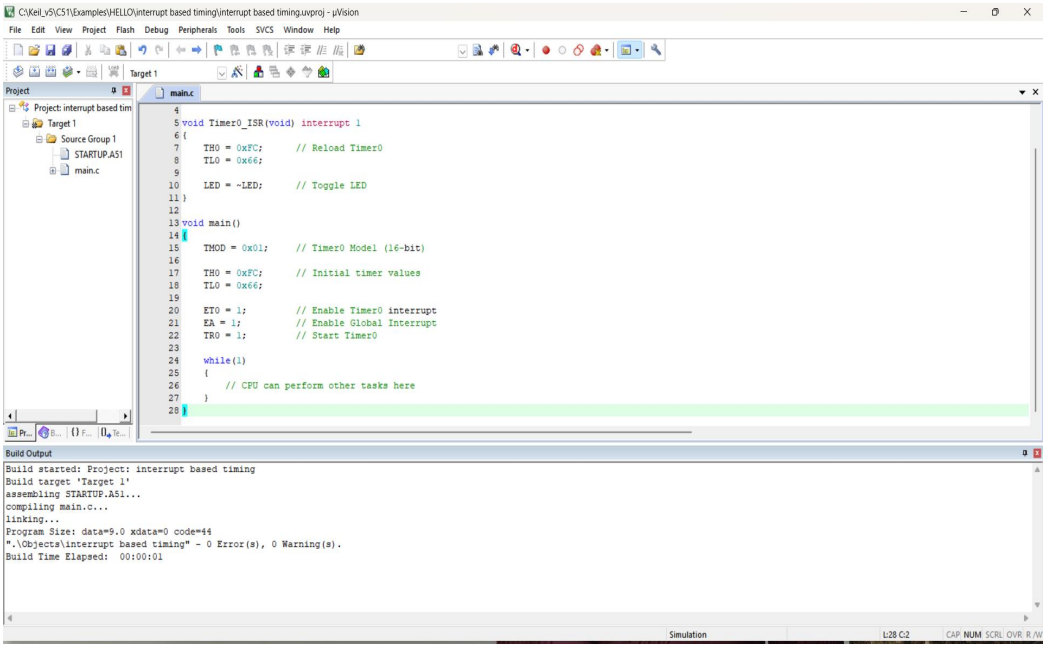


Fig 3: Timer0 interrupt-based program – main.c in Keil µVision, build successful (0 Errors, 0 Warnings)

6.4 Program 2 – Simulation / Debug Run

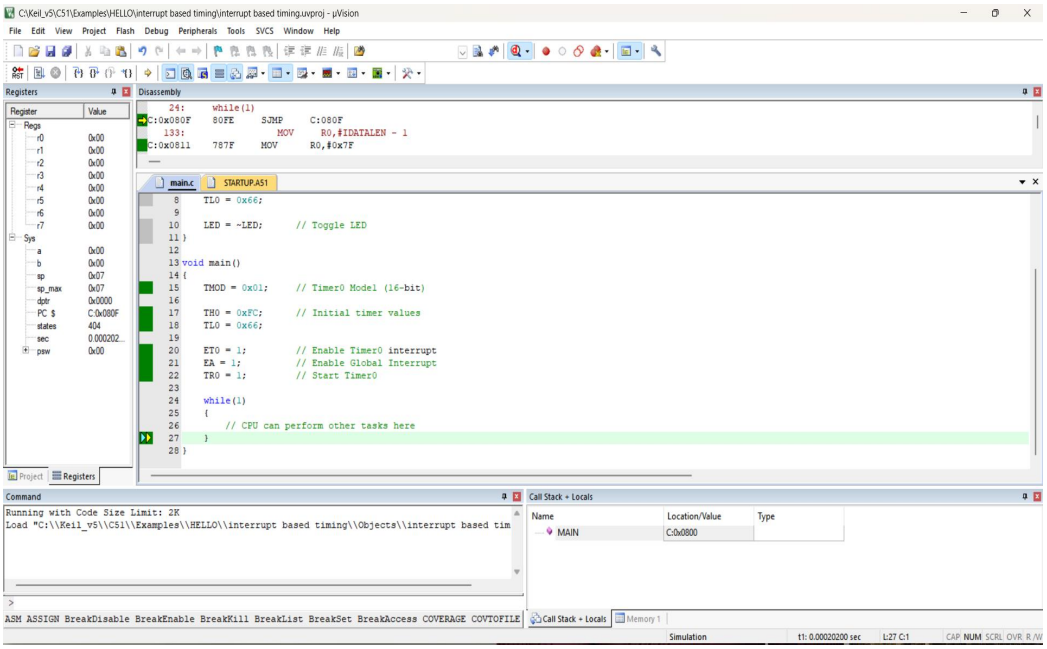


Fig 4: Debug session showing while(1) idle loop in main() while Timer0 ISR (interrupt 1) handles LED toggling in the background

7. Comparison / Competitive Performance Analysis

Polling-Based Delay	Interrupt-Based Timing
Delay created using nested software loops	Delay generated by Timer0 hardware overflow
CPU is blocked/busy during the entire delay	CPU is free to execute other tasks while waiting
Timing depends on instruction cycle counting; less precise	Timing is hardware-driven and highly accurate
Simple to implement, no interrupt setup needed	Requires timer and interrupt configuration
Not scalable to multiple time-based tasks	Can be extended to handle multiple time-based tasks concurrently
Less suitable for real-time systems	Ideal for real-time embedded/industrial systems

8. Conclusion

Both programs achieve the same functional result of toggling an LED at a fixed rate, but they differ fundamentally in how the CPU's time is used. The polling-based approach is simple to write and understand, but it wastes CPU cycles by keeping the processor busy inside the delay loop, and its timing accuracy is sensitive to compiler optimizations and instruction timing. The interrupt-based approach, using Timer0 in Mode 1, offloads the timing function to dedicated hardware and frees the CPU to perform other tasks in main() while still toggling the LED at a precise, hardware-controlled interval. For real-time, industrial, and multi-tasking embedded applications, interrupt-driven timing is therefore the preferred and more efficient design choice over polling-based delays.